

separação da interface de implementação 489
set, função 481
solicitação de um serviço de um objeto 472
<string>, arquivo de cabeçalho 476
string, classe 476
string vazia 481
substr, função-membro da classe string 495

tipo de retorno 473
tipo definido pelo usuário 474
UML, diagrama de classes 474
validação 494
variáveis locais 477
verificação de validade 494
void, tipo de retorno 473

■ Exercícios de autorrevisão

16.1 Preencha os espaços em cada uma das sentenças:

- a) Uma casa é para uma planta como um(a) _____ é para uma classe.
- b) Toda definição de classe contém a palavra-chave _____ seguida imediatamente pelo nome da classe.
- c) Uma definição de classe normalmente é armazenada em um arquivo com a extensão de nome de arquivo _____.
- d) Cada parâmetro em um cabeçalho de função deve especificar tanto um(a) _____ quanto um(a) _____.
- e) Quando cada objeto de uma classe mantém sua própria cópia de um atributo, a variável que representa o atributo também é conhecida como um _____.
- f) A palavra-chave `public` é um(a) _____.
- g) O tipo de retorno _____ indica que uma função realizará uma tarefa, mas não retornará nenhuma informação quando completar sua tarefa.
- h) A função _____ da biblioteca <string> lê caracteres até que um caractere de newline seja encontrado, e depois copia esses caracteres para a string especificada.
- i) Quando uma função-membro é definida fora da definição de classe, o cabeçalho de função deve incluir o nome da classe e o(a) _____, seguido(a) pelo nome da função para 'amarrar' a função-membro à definição da classe.
- j) O arquivo de código-fonte e quaisquer outros arquivos que usem uma classe podem incluir o arquivo de cabeçalho da classe por meio de uma diretiva de pré-processador _____.

16.2 Indique se cada um dos itens a seguir é *verdadeiro* ou *falso*. Justifique sua resposta em casos de alternativas falsas.

- a) Por convenção, os nomes de função começam com uma letra maiúscula, e todas as palavras seguintes no nome começam com uma letra maiúscula.
- b) Os parênteses vazios após um nome de função em um protótipo de função indicam que a função não requer nenhum parâmetro para realizar sua tarefa.
- c) Os dados-membro ou funções-membro declaradas com o especificador de acesso `private` são acessíveis às funções-membro da classe em que são declaradas.
- d) Variáveis declaradas no corpo de uma função-membro em particular são conhecidas como dados-membro, e podem ser usadas em todas as funções-membro da classe.
- e) O corpo de toda função é delimitado pelas chaves esquerda e direita (`{` e `}`).
- f) Qualquer arquivo de código-fonte que contenha `int main()` pode ser usado para executar um programa.
- g) Os tipos de argumentos em uma chamada de função devem ser coerentes com os tipos dos parâmetros correspondentes na lista de parâmetros do protótipo de função.

16.3 Qual é a diferença entre uma variável local e um dado-membro?

16.4 Explique a finalidade de um parâmetro de função. Qual é a diferença entre um parâmetro e um argumento?

■ Respostas dos exercícios de autorrevisão

16.1 a) objeto. b) `class`. c) `.h`. d) tipo, nome. e) dado-membro. f) especificador de acesso. g) `void`. h) `getline`. i) operador binário de resolução de escopo (`::`). j) `#include`.

16.2 a) Falso. Nomes de função começam com uma letra minúscula e todas as palavras subsequentes começam com uma letra maiúscula. b) Verdadeiro. c) Verdadeiro.

d) Falso. Essas variáveis são variáveis locais, e só podem ser usadas na função-membro em que forem declaradas. e) Verdadeiro. f) Verdadeiro. g) Verdadeiro.

16.3 Uma variável local é declarada no corpo de uma função, e só pode ser usada a partir do ponto em que ela é declarada até a chave final do bloco em que é declarada. Um

dado-membro é declarado em uma classe, mas não no corpo de qualquer uma das funções-membro da classe. Cada objeto de uma classe tem uma cópia separada dos dados-membro da classe. Os dados-membro são acessíveis a todas as funções-membro da classe.

- 16.4** Um parâmetro representa informações adicionais que uma função requer para realizar sua tarefa. Cada

parâmetro exigido por uma função é especificado no cabeçalho da função. Um argumento é o valor fornecido na chamada da função. Quando a função é chamada, o valor do argumento é passado para o parâmetro da função, de modo que a função possa realizar sua tarefa.

Exercícios

- 16.5** Explique a diferença entre um protótipo de função e uma definição de função.
- 16.6** O que é um construtor default? Como os dados-membro de um objeto são inicializados se uma classe tiver apenas um construtor default definido implicitamente?
- 16.7** Explique a finalidade de um dado-membro.
- 16.8** O que é um arquivo de cabeçalho? O que é um arquivo de código-fonte? Discuta a finalidade de cada um.
- 16.9** Explique como um programa poderia usar a classe `string` sem inserir uma declaração `using`.
- 16.10** Explique por que uma classe poderia oferecer uma função `set` e uma função `get` para um dado-membro.
- 16.11 Modificando a classe `GradeBook`.** Modifique a classe `GradeBook` (figuras 16.11 e 16.12) da seguinte forma:
- Inclua um segundo dado-membro `string` que represente o nome do instrutor do curso.
 - Forneça uma função `set` para mudar o nome do instrutor e uma função `get` para recuperá-lo.
 - Modifique o construtor para especificar os parâmetros de nome do curso e nome do instrutor.
 - Modifique a função `displayMessage` para mostrar uma mensagem de boas-vindas e o nome do curso, depois a string “Esse curso é apresentado por: ”, seguido pelo nome do instrutor.

Use a sua classe modificada em um programa de teste que demonstre as novas capacidades da classe.

- 16.12 Classe `Account`.** Crie uma classe `Account` que um banco poderia usar para representar as contas bancárias dos clientes. Inclua um dado-membro do tipo `int` para representar o saldo da conta. Forneça um construtor que receba um saldo inicial e use-o para inicializar o dado-membro. O construtor deverá validar o saldo inicial para garantir que ele seja maior ou igual a 0. Se não for, defina o saldo como 0 e exiba uma mensagem de erro que mostre que o saldo inicial era inválido. Forneça três funções-membro. A função-membro `credit` deve somar um valor ao saldo atual. A função-membro `debit` deve retirar

dinheiro da `Account` e garantir que o valor do débito não exceda o saldo de `Account`. Se exceder, o saldo deve ser deixado inalterado, e a função deve imprimir uma mensagem que mostre “Valor do débito superior ao saldo da conta”. A função-membro `getBalance` deverá retornar o saldo atual. Crie um programa que gere dois objetos `Account` e teste as funções-membro da classe `Account`.

- 16.13 Classe `Fatura`.** Crie uma classe chamada `Fatura` que uma loja de ferragens poderia usar para representar uma fatura para um item vendido na loja. Uma `Fatura` deverá incluir quatro membros — um número de peça (tipo `string`), uma descrição da peça (tipo `string`), a quantidade do item sendo comprado (tipo `int`) e o preço por item (tipo `int`). Sua classe deverá ter um construtor que inicialize os quatro dados-membro. Forneça uma função `set` e uma função `get` para cada dado-membro. Além disso, forneça uma função-membro chamada `obterValorFatura` que calcule o valor da fatura (ou seja, multiplique a quantidade pelo preço por item), depois retorne o valor como um valor `int`. Se a quantidade não for positiva, ela deve ser definida como 0. Se o preço por item não for positivo, ele deve ser definido como 0. Escreva um programa de teste que demonstre as capacidades da classe `Fatura`.

- 16.14 Classe `Empregado`.** Crie uma classe chamada `Empregado` que inclua três partes de informação como dados-membro — um primeiro nome (tipo `string`), um sobrenome (tipo `string`) e um salário mensal (tipo `int`). Sua classe deverá ter um construtor que inicialize os três dados-membro. Forneça uma função `set` e uma função `get` para cada dado-membro. Se o salário mensal não for positivo, defina-o como 0. Escreva um programa de teste que demonstre as capacidades da classe `Empregado`. Crie dois objetos `Empregado` e mostre o salário *anual* de cada objeto. Depois, dê a cada `Empregado` um aumento de 10 por cento e exiba o salário anual do `Empregado` novamente.

- 16.15 Classe `Date`.** Crie uma classe chamada `Date` que inclua três partes de informação como dados-membro — um

mês (tipo `int`), um dia (tipo `int`) e um ano (tipo `int`). Sua classe deverá ter um construtor com três parâmetros que sejam usados para inicializar os três dados-membro. Para a finalidade desse exercício, suponha que os valores fornecidos para o ano e para o dia estejam corretos, mas garanta que o valor do mês esteja no intervalo de 1 a 12;

se não estiver, defina o mês como 1. Forneça uma função *set* e *get* para cada dado-membro. Forneça uma função-membro *mostraData* que apresente o mês, o dia e o ano separados por barras (/). Escreva um programa de teste que demonstre as capacidades da classe `Date`.

Fazendo a diferença

16.16 Calculadora da frequência cardíaca. Enquanto estiver se exercitando, você pode usar um monitor de frequência cardíaca para ver se sua taxa de batimentos está dentro de uma faixa segura, sugerida por seus treinadores e médicos. De acordo com a American Heart Association (AHA) (<www.americanheart.org/presenter.jhtml?identifier=4736>), a fórmula para calcular a *frequência cardíaca máxima* em batimentos por minuto é 220 menos sua idade em anos. Sua *frequência cardíaca ideal* está em uma faixa entre 50-85 por cento da sua frequência máxima. [Nota: *essas fórmulas são estimativas fornecidas pela AHA. As frequências cardíacas máxima e ideal podem variar com base na saúde, condição física e sexo do indivíduo. Consulte sempre um médico ou um profissional de saúde qualificado antes de iniciar ou modificar um programa de exercícios.*] Crie uma classe chamada `FrequenciaCardiaca`. Os atributos da classe deverão incluir o primeiro nome, o sobrenome e data de nascimento da pessoa (essa última deve ter atributos separados para dia, mês e ano). Sua classe deverá ter um construtor que receba esses dados como parâmetros. Para cada atributo, forneça funções *set* e *get*. A classe também deverá incluir uma função *obterIdade* que calcule e retorne a idade da pessoa (em anos), uma função *obterFrequenciaMaxima* que calcule e retorne a frequência cardíaca máxima da pessoa e uma função *obterFrequenciaIdeal*, que calcule e retorne a frequência cardíaca ideal da pessoa. Como você ainda não sabe como obter a data atual do computador, a função *obterIdade* deve pedir que o usuário digite o dia, o mês e o ano atuais antes de calcular a idade da pessoa. Escreva uma aplicação que peça a informação da pessoa, instancie um objeto da classe `FrequenciasCardiacas` e imprima a informação desse objeto — incluindo nome, sobrenome e data de nascimento —, e depois calcule e imprima a idade da pessoa (em anos),

sua frequência cardíaca máxima e sua frequência cardíaca ideal.

16.17 Informatização de registros de saúde. Ultimamente, a questão da informatização dos registros de saúde tem aparecido muito nos jornais. Essa possibilidade está sendo abordada com cautela devido a questões de privacidade e segurança de dados confidenciais, entre outras. [Trataremos essas questões em outros exercícios.] A informatização de registros de saúde poderia tornar mais fácil o compartilhamento de perfis e históricos de saúde de pacientes com todos os profissionais de saúde que cuidam deles. Isso poderia melhorar a qualidade do atendimento, evitar conflitos entre medicamentos e poderia evitar a prescrição de medicamentos inapropriados, além de reduzir custos e, em emergências, salvar vidas. Nesse exercício, você projetará uma classe `PerfilSaude` 'inicial' para uma pessoa. Os atributos da classe devem incluir nome, sobrenome, gênero e data de nascimento (em atributos separados para dia, mês e ano de nascimento), altura (em centímetros) e peso (em quilos). Sua classe deverá ter um construtor que receba esses dados. Para cada atributo, forneça funções *set* e *get*. A classe também deverá incluir funções que calculem e retornem a idade do usuário em anos, as frequências cardíacas máxima e ideal (ver Exercício 16.16) e o índice de massa corporal (IMC; ver Exercício 2.32). Escreva uma aplicação que peça a informação da pessoa, instancie um objeto da classe `PerfilSaude` para essa pessoa e imprima a informação desse objeto — incluindo nome, sobrenome, sexo, data de nascimento, altura e peso —, e depois calcule e imprima a idade da pessoa em anos, IMC, frequência cardíaca máxima e intervalo da frequência ideal. A aplicação também deverá mostrar o gráfico dos 'valores de IMC' do Exercício 2.32. Use a mesma técnica do Exercício 16.16 para calcular a idade da pessoa.